

ENTREGA SPRINT

FEDERICO URBINA

Curso especialización

IT ACADEMY



Nivel 1

Descarga los archivos CSV, estudiales y diseña una base de datos con un esquema de estrella que contenga, al menos 4 tablas de las que puedas realizar las siguientes consultas:

Operaciones realizadas:

Creo una nueva BBDD llamada Sprint04

```
1  -- Creo la BBDD
2  • CREATE DATABASE sprint04;
3  • USE sprint04;
4
```

ut

Action Output ▼

	Time	Action
1	14:31:23	CREATE DATABASE sprint04
2	14:31:46	USE sprint04

Operaciones realizadas:

- Revisé los archivos CSV para identificar los campos disponibles.
- Creé la base de datos sprint04.
- Creé las tablas users, companies, credit_cards, products y transactions.
- Definí una única tabla users para cargar los datos de american_users.csv y european_users.csv.
- Definí claves primarias en cada tabla según el identificador de los datos.

```

7 • CREATE TABLE users (
8     id VARCHAR(250) NOT NULL,
9     name VARCHAR(250),
10    surname VARCHAR(250),
11    phone VARCHAR(250),
12    email VARCHAR(250),
13    birth_date VARCHAR(250),
14    country VARCHAR(250),
15    city VARCHAR(250),
16    postal_code VARCHAR(250),
17    address VARCHAR(250),
18    PRIMARY KEY (id)
19 );
20
21 • CREATE TABLE companies (
22     company_id VARCHAR(250) NOT NULL,
23     company_name VARCHAR(250),
24     phone VARCHAR(250),
25     email VARCHAR(250),
26     country VARCHAR(250),
27     website VARCHAR(250),
28     PRIMARY KEY (company_id)
29 );
30
31 • CREATE TABLE credit_cards (
32     id VARCHAR(250) NOT NULL,
33     user_id VARCHAR(250),
34     iban VARCHAR(250),
35     pan VARCHAR(250),
36     pin VARCHAR(250),
37     cvv VARCHAR(250),
38     track1 VARCHAR(250),
39     track2 VARCHAR(250),
40     expiring_date VARCHAR(250),
41     PRIMARY KEY (id)
42 );

```

```

43
44 • CREATE TABLE products (
45     id VARCHAR(250) NOT NULL,
46     product_name VARCHAR(250),
47     price VARCHAR(250),
48     colour VARCHAR(250),
49     weight VARCHAR(250),
50     warehouse_id VARCHAR(250),
51     PRIMARY KEY (id)
52 );
53
54 • CREATE TABLE transactions (
55     id VARCHAR(250) NOT NULL,
56     card_id VARCHAR(250),
57     business_id VARCHAR(250),
58     timestamp VARCHAR(250),
59     amount VARCHAR(250),
60     declined VARCHAR(250),
61     product_ids VARCHAR(250),
62     user_id VARCHAR(250),
63     lat VARCHAR(250),
64     longitude VARCHAR(250),
65     PRIMARY KEY (id)
66 );
67

```

Output

#	Time	Action
1	14:47:22	CREATE TABLE users (id VARCHAR(250) NOT NU
2	14:47:55	CREATE TABLE companies (company_id VARCHA
3	14:48:23	CREATE TABLE products (id VARCHAR(250) NOT
4	14:48:32	CREATE TABLE transactions (id VARCHAR(250) N


```

74 • LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/companies 2.csv'
75 INTO TABLE companies
76 FIELDS TERMINATED BY ','
77 LINES TERMINATED BY '\r\n'
78 IGNORE 1 LINES
79 (company_id, company_name, phone, email, country, website);
80
81 • SELECT count(1) FROM companies;
82
83 •

```

100% 60:79

Action Output

	Time	Action
9	12:44:28	SELECT * FROM sprint04.companies LIMIT 0, 50000

```

83 • LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/american_users.csv'
84 INTO TABLE users
85 FIELDS TERMINATED BY ','
86 ENCLOSED BY '"'
87 LINES TERMINATED BY '\n'
88 IGNORE 1 LINES
89 (id, name, surname, phone, email, birth_date, country, city, postal_code, address);
90
91 • SELECT COUNT(1) FROM users; -- Verifico si están los
92
93 •

```

100% 55:91

Result Grid

COUNT(1)
1010

Result 1

Action Output

	Time	Action	Response
1	12:59:16	LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/american_users.csv' INTO TABLE users FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES...	1010 row(s) a...
2	13:02:47	SELECT COUNT(1) FROM users LIMIT 0, 50000	1 row(s) retur...

```

93 • LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/european_users.csv'
94 INTO TABLE users
95 FIELDS TERMINATED BY ','
96 ENCLOSED BY '"'
97 LINES TERMINATED BY '\n'
98 IGNORE 1 LINES
99 (id, name, surname, phone, email, birth_date, country, city, postal_code, address);
100
101 • SELECT COUNT(1) FROM users; -- 5000 usuarios es la suma de los 1010 Americanos + 3990 de Europeos
102

```

100% 98:101

Result Grid

COUNT(1)
5000

Result 2

Action Output

	Time	Action	Response
3	13:06:17	LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/european_users.csv' INTO TABLE users FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES...	3990 row(s)...
4	13:06:56	SELECT COUNT(1) FROM users LIMIT 0, 50000	1 row(s) retur...


```

103 • LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/credit_cards.csv'
104 INTO TABLE credit_cards
105 FIELDS TERMINATED BY ','
106 ENCLOSED BY '"'
107 LINES TERMINATED BY '\n'
108 IGNORE 1 LINES;
109
110 • SELECT COUNT(1) FROM credit_cards;
111

```

00% 35:110

Result Grid Filter Rows: Search Export:

COUNT(1)
5000

Result 3

Action Output

	Time	Action	Response
5	13:12:26	LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/credit_cards.csv' INTO TABLE credit_cards FIELDS TERMINATED BY ',' ENCLOSED BY '"' LI...	5000 row(s)...
6	13:12:34	SELECT COUNT(1) FROM credit_cards LIMIT 0, 50000	1 row(s) retur...

```

112 • LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/products.csv'
113 INTO TABLE products
114 FIELDS TERMINATED BY ','
115 ENCLOSED BY '"'
116 LINES TERMINATED BY '\n'
117 IGNORE 1 LINES;
118
119 • SELECT COUNT(*) AS Prod FROM products;
120

```

100% 39:119

Result Grid Filter Rows: Search Export:

Prod
100

Result 4

Action Output

	Time	Action	Response
7	13:15:14	LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/products.csv' INTO TABLE products FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TE...	100 row(s) af...
8	13:15:30	SELECT COUNT(*) AS Prod FROM products LIMIT 0, 50000	1 row(s) retur...

```

121 • LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/transactions.csv'
122 INTO TABLE transactions
123 FIELDS TERMINATED BY ';'
124 ENCLOSED BY '"'
125 LINES TERMINATED BY '\n'
126 IGNORE 1 LINES;
127
128 • SELECT COUNT(*) AS TOTAL_TRANSACTIONS
129 FROM transactions;
130

```

100% 19:129

Result Grid Filter Rows: Search Export:

TOTAL_TRANSACTIONS
100000

Result 5

Action Output

	Time	Action	Response
1	13:25:03	LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/transactions.csv' INTO TABLE t...	100000 row(s)...
2	13:25:08	SELECT COUNT(*) AS TOTAL_TRANSACTIONS FROM transactions LIMIT 0, 50000	1 row(s) retur...

max_len_id	max_len_name	max_len_phone	max_len_email	max_len_country	max_len_website	total_filas
6	35	14	38	14	31	100

```

143 • ALTER TABLE companies
144     MODIFY company_id VARCHAR(20),
145     MODIFY company_name VARCHAR(50),
146     MODIFY phone VARCHAR(20),
147     MODIFY email VARCHAR(50),
148     MODIFY country VARCHAR(20),
149     MODIFY website VARCHAR(100);
150
100% 31:149

```

Action Output

	Time	Action	Response
21	21:48:21	ALTER TABLE companies MODIFY company_id VARCHAR(20), MODIFY company_name VARCHAR(50),	

name	surname	phone	email	postal_code	birth_date	max_len_email	country	address	total_filas
9	11	15	40	8	12	40	14	33	5000

```

172 -----| cambio el string al formato que llevará el formato DATE
173 • UPDATE users
174     SET birth_date = STR_TO_DATE(birth_date, '%b %d, %Y');
175
176 -----| compruebo el cambio
177 • SELECT birth_date
178     FROM users
179     LIMIT 5;
180
181 • ALTER TABLE users
182     MODIFY id VARCHAR(20),
183     MODIFY name varchar(20),
184     MODIFY surname varchar(25),
185     MODIFY email VARCHAR(50),
186     MODIFY birth_date DATE,
187     MODIFY country VARCHAR(20),
188     MODIFY city VARCHAR(20),
189     MODIFY postal_code VARCHAR(10),
190     MODIFY address VARCHAR(50);
191
100% 30:190

```

Action Output

	Time	Action	Response
24	21:51:23	UPDATE users SET birth_date = STR_TO_DATE(birth_date, '%b %d, %Y')	5000 row(s) affected
25	21:51:55	SELECT birth_date FROM users LIMIT 5	5 row(s) returned
26	21:52:20	ALTER TABLE users MODIFY id VARCHAR(20), MODIFY name varchar(20), MODIFY surname varchar(25), MO...	5000 row(s) affected

user_id	iban	pan	pin	cvv	track1	track2	expiring_date	total_filas
4	31	19	4	3	52	36	8	5000

```

-----| cambio el string al formato que llevará el formato DATE
UPDATE credit_cards
    SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y');
-----| verifico el cambio
214 • SELECT expiring_date
215     FROM credit_cards
216     LIMIT 5;
217
218
219 • ALTER TABLE credit_cards
220     MODIFY id VARCHAR(20),
221     MODIFY user_id VARCHAR(20),
222     MODIFY iban VARCHAR(34),
223     MODIFY pan VARCHAR(19),
224     MODIFY cvv VARCHAR(4),
225     MODIFY pin VARCHAR(4),
226     MODIFY track1 VARCHAR(60),
227     MODIFY track2 VARCHAR(60),
228     MODIFY expiring_date DATE;
229
100% 1:211

```

Action Output

	Time	Action	Response
29	21:56:27	UPDATE credit_cards SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y')	5000 row(s) affected
30	21:56:32	SELECT expiring_date FROM credit_cards LIMIT 5	5 row(s) returned
31	21:56:46	ALTER TABLE credit_cards MODIFY id VARCHAR(20), MODIFY user_id VARCHAR(20), MODIFY iban VARCHAR(...)	5000 row(s) affected

card_id	business_id	time_stamp	amount	declined	product_ids	user_id	lat	longitude
8	6	19	6	1	19	4	18	19

```

247 • ALTER TABLE transactions
248     MODIFY id VARCHAR(50),
249     MODIFY card_id VARCHAR(20),
250     MODIFY business_id VARCHAR(20),
251     MODIFY product_ids VARCHAR(100),
252     MODIFY user_id VARCHAR(20),
253     -- Usamos CHANGE para renombrar y pasar a DATETIME a la vez
254     CHANGE timestamp time_stamp DATETIME,
255     MODIFY amount DECIMAL(10,2),
256     MODIFY declined TINYINT,
257     MODIFY lat DOUBLE,
258     MODIFY longitude DOUBLE;
259
260 •
100% 27:258

```

Action Output

	Time	Action	Response
4	22:18:31	SELECT COUNT(*) AS TOTAL_TRANSACTIONS FROM transactions LIMIT 0, 50000	1 row(s) re
5	22:18:43	ALTER TABLE transactions MODIFY id VARCHAR(50), MODIFY card_id VARCHAR(20), MODIFY bus...	100000 row

product_name	price	colour	weight	warehouse_id
29	7	7	3	6

```

267 -----|Verifico los datos
268 • SELECT price, weight
269     FROM products
270     LIMIT 5;
271
272 -----| Hay q quitar el dolar
273 • UPDATE products
274     SET price = REPLACE(price, '$', '');
275
276 -----| Verifico
277 • SELECT price, weight
278     FROM products
279     LIMIT 5;
280
281 • ALTER TABLE products
282     MODIFY id VARCHAR(20),
283     MODIFY product_name VARCHAR(50),
284     MODIFY price DECIMAL(10,2),
285     MODIFY colour VARCHAR(15),
286     MODIFY weight DECIMAL(10,2),
287     MODIFY warehouse_id VARCHAR(10);
288
100% 47:261

```

Action Output

	Time	Action	Response
10	22:22:27	SELECT price, weight FROM products LIMIT 5	
11	22:22:50	ALTER TABLE products MODIFY id VARCHAR(20), MODIFY product_name VARCHAR(50), MODIFY price DECIM...	

```

131 • ALTER TABLE credit_cards
132     ADD CONSTRAINT fk_credit_cards_users
133     FOREIGN KEY (user_id) REFERENCES users(id);
134
135 • ALTER TABLE transactions
136     ADD CONSTRAINT fk_transactions_users
137     FOREIGN KEY (user_id) REFERENCES users(id),
138     ADD CONSTRAINT fk_transactions_cards
139     FOREIGN KEY (card_id) REFERENCES credit_cards(id),
140     ADD CONSTRAINT fk_transactions_companies
141     FOREIGN KEY (business_id) REFERENCES companies(company_id);
142
143 -----| Ejercicio
100% 62:141

```

Action Output

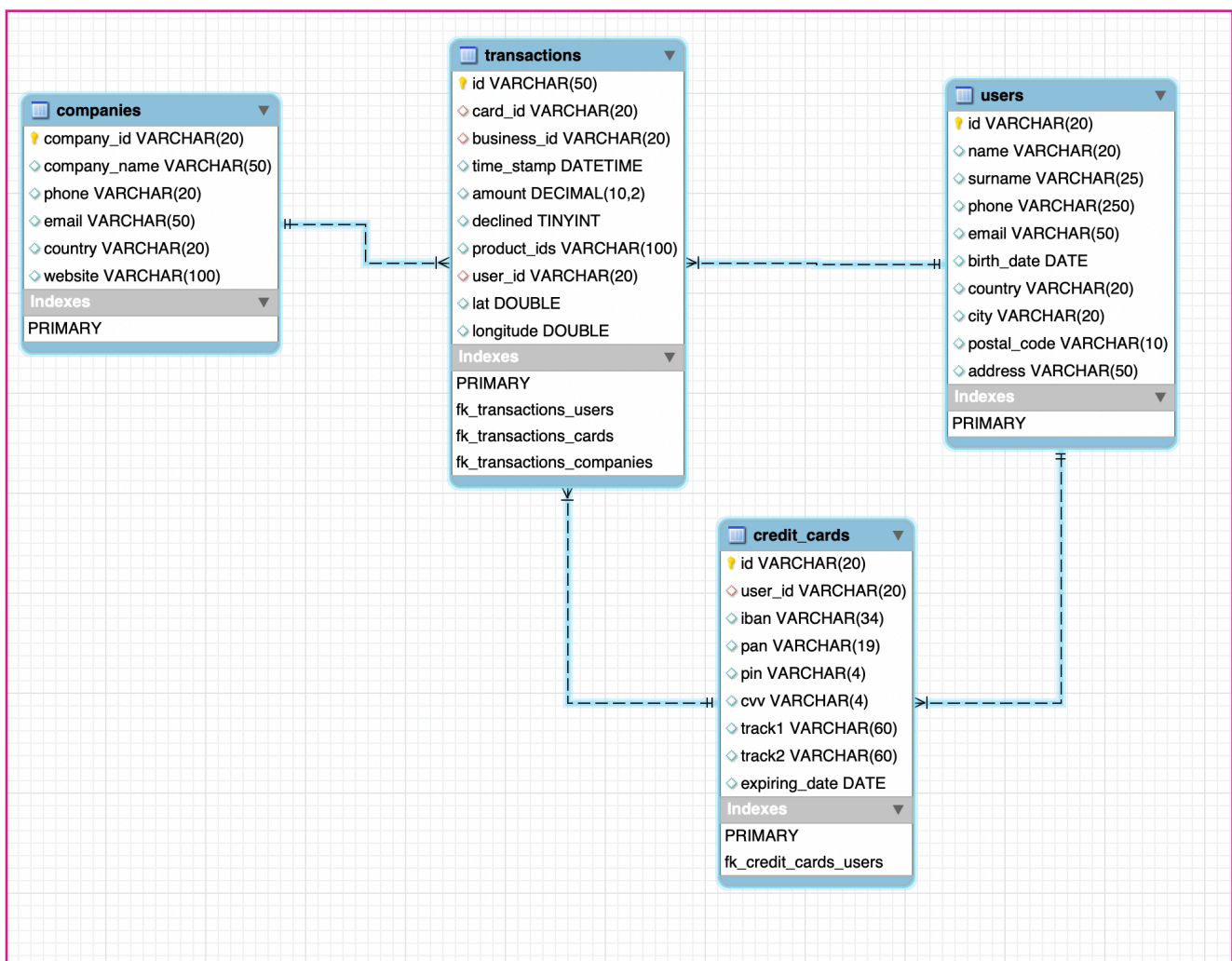
	Time	Action	Response
7	12:26:34	ALTER TABLE credit_cards ADD CONSTRAINT fk_credit_cards_...	5000 row(s)...
8	12:26:45	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_...	100000 row(s)...

Operaciones realizadas:

- Importé los datos de los archivos .CSV mediante sentencias LOAD DATA LOCAL INFILE.
- En el caso de users, cargué dos archivos CSV en una misma tabla comprobando q quedaba la suma de los 2
- Verifiqué la correcta carga de cada tabla mediante consultas SELECT COUNT(*).
- Modifiqué los tipos en las diferentes entidades y campos ajustando para optimizar.
- Creé entidad referencial entre tablas.

Resultado clave:

- Todas las tablas contienen los datos importados desde los archivos CSV correspondientes.



Operaciones realizadas:

- Usé una subconsulta sobre la tabla transactions.
- Agrupé las transacciones por user_id.
- Filtré los usuarios que tienen más de 80 transacciones.
- Seleccioné los registros correspondientes en la tabla users.

Resultado clave:

- Se obtienen los usuarios que han realizado más de 80 transacciones.

Ejercicio 1

Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.

300 -----| Ejercicio 1.1
301
302 • **SELECT ***
303 **FROM** users
304 **WHERE** id **IN** (
305 **SELECT** user_id
306 **FROM** transactions
307 **GROUP BY** user_id
308 **HAVING** COUNT(*) > 80
309);
310

100% 3:299

Result Grid Filter Rows: Search Edit: Export/Import:

id	name	surname	phone	email	birth_date	country	city	postal_code	address
185	Molly	Gilliam	0800 120 8023	donec@outlook.couk	1993-12-21	United Kingdom	London	EC1A 1BB	P.O. Box 202, 5638 Mi Rd.
289	Dxwgi	Hwcru	+98-309-8797	dxwgi.hwcru@example.com	1976-08-20	Germany	Stuttgart	70173	82 Hwcru Street
318	Bnyr	Astuw	+33-120-9644	bnyr.astuw@example.com	1974-05-03	Italy	Genoa	16100	53 Astuw Street
454	Sfzzoh	Xgvfridxs	+58-495-3945	sfzzoh.xgvfridxs@example.com	1962-08-28	Poland	Gdansk	80-001	52 Xgvfridxs Street

users 44

Action Output

	Time	Action	Response
3	22:38:01	SELECT * FROM users WHERE id IN (SELECT user_id FROM transactions GROUP BY user_id HAVING COU...	4 row(s) returned

Ejercicio 2

Muestra la media de amount por IBAN de las tarjetas de crédito en la compañía Donec Ltd., utiliza por lo menos 2 tablas.

```
156 • SELECT cc.iban,
157         ROUND(AVG(t.amount),2) AS media_amount
158 FROM transactions t
159 JOIN credit_cards cc
160     ON t.card_id = cc.id
161 JOIN companies co
162     ON t.business_id = co.company_id
163 WHERE co.company_name = 'Donec Ltd'
164 GROUP BY cc.iban;
165
```

100% 66:154

Result Grid

iban	media_amount
XX776752917845952975555640	257.37
XX413827362289719304908990	139.59
XX347787246070769610780308	240.41
XX688768436543090894854602	188.58
MK28368851538688349	439.39

Result 11

Action Output

	Time	Action	Response
✓ 11	12:35:08	SELECT cc.iban, ROUND(AVG(t.amount),2) AS media_amount...	371 row(s) re...

Operaciones realizadas:

- Se relaciona la tabla transactions con credit_cards mediante el identificador de tarjeta.
- Se relaciona la tabla transactions con companies mediante el identificador de empresa.
- Se filtran las transacciones correspondientes a la empresa Donec Ltd..
- Se agrupan los resultados por

IBAN y se calcula la media del campo amount.

Resultado clave:

- Se obtiene la media del importe de las transacciones para cada IBAN asociado a la empresa Donec Ltd..



Nivel 2

Crea una nueva tabla que refleje el estado de las tarjetas de crédito basado en *si las tres últimas transacciones han sido declinadas entonces es inactivo, si al menos una no es rechazada entonces es activo*. Partiendo de esta tabla responde:

Ejercicio 1

¿Cuántas tarjetas están activas?

```

326 • SHOW FULL TABLES;
327
328 • CREATE TABLE credit_card_status AS
329 WITH ranked_transactions AS (
330     SELECT
331         id,
332         card_id,
333         declined,
334         time_stamp,
335         ROW_NUMBER() OVER (
336             PARTITION BY card_id
337             ORDER BY time_stamp DESC
338         ) AS row_num
339     FROM transactions
340 )
341 SELECT
342     id,
343     card_id,
344     declined,
345     time_stamp,
346     row_num
347 FROM ranked_transactions
348 WHERE row_num IN (1, 2, 3);
349
350 • ALTER TABLE credit_card_status
351     ADD CONSTRAINT fk_CCStatus_credit_cards
352     FOREIGN KEY (card_id)
353     REFERENCES credit_cards(id);
354
355 -- Compruebo los datos de la tabla credit_card_status
356 • SELECT

```

Action	Time	Action	Response
1	22:44:09	SHOW FULL TABLES	5 row(s) returned
2	22:44:16	CREATE TABLE credit_card_status AS WITH ranked_transactions AS (SELECT id, card_id, declined, time_stamp, ROW_NUMBER() OVER (PARTITION BY card_id ORDER BY time_stamp DESC) AS row_num FROM transactions) SELECT id, card_id, declined, time_stamp, row_num FROM ranked_transactions WHERE row_num IN (1, 2, 3);	15000 row(s) affected Records: 15000 Duplicates: 0 Warnings: 0

```

360 -- -----| ¿Cuántas tarjetas están activas?
361 • SELECT COUNT(card_id) AS 'Núm. tarjetas activas'
362 FROM (
363     SELECT card_id
364     FROM credit_card_status
365     GROUP BY card_id
366     HAVING SUM(declined) < 3
367 ) AS credit_card_active;
368
369

```

Núm. tarjetas activas
4995

Result 52

Action	Time	Action	Response
4	22:47:30	SELECT * FROM credit_card_status LIMIT 0, 50000	15000 row(s) returned
5	22:47:50	SELECT COUNT(card_id) AS 'Núm. tarjetas activas' FROM (SELECT card_id FROM credit_card_status GROUP BY card_id HAVING SUM(declined) < 3) AS credit_card_active;	1 row(s) returned
6	22:48:48	SELECT COUNT(card_id) AS 'Núm. tarjetas activas' FROM (SELECT card_id FROM credit_card_status GROUP BY card_id HAVING SUM(declined) < 3) AS credit_card_active;	1 row(s) returned

El ejercicio solicita crear una nueva tabla que refleje el estado de las tarjetas de crédito en función de sus tres últimas transacciones.

La tabla representa una instantánea del estado de las tarjetas en un momento determinado, ya que cualquier transacción posterior no modifica los datos almacenados.

Para resolver el ejercicio, es necesario analizar el resultado (declined) de las tres transacciones más recientes asociadas a cada tarjeta y clasificar su estado como activa o inactiva según el criterio indicado.

En un entorno real, esta información podría obtenerse mediante una vista o se podría añadir a la tabla un proceso automático de actualización con triggers para mantenerla siempre actualizada.

En este caso, implemento la solución solicitada en el enunciado.

Operaciones realizadas:

- Ordené las transacciones por tarjeta y fecha utilizando ROW_NUMBER().
- Identifiqué las tres últimas transacciones de cada tarjeta.
- Creé la tabla credit_card_status con dichas transacciones para determinar el estado de cada tarjeta.
- Teniendo en cuenta la nueva tabla de “status” conté la cantidad de tarjetas Activas.

Resultado Clave:

- La tabla credit_card_status contiene las tres últimas transacciones de cada tarjeta y permite identificar su estado según el valor de declined.
- Dar respuesta a la consulta.



Nivel 3

Crea una tabla con la que podamos unir los datos del nuevo archivo products.csv con la base de datos creada, teniendo en cuenta que desde transaction tienes product_ids. Genera la siguiente consulta:



Ejercicio 1

Necesitamos conocer el número de veces que se ha vendido cada producto.

Operaciones realizadas:

- Cree tabla intermedia para poder vincular transacciones y productos
- Cree relación
- incluí los datos en la tabla intermedia valiéndome de tabla Jason para poder separar los productos que originalmente estaban incluidos en una lista

```

-----| Ejercicio 3.1
370 • SHOW FULL TABLES;
371 -----Creación de a tabla
372 • CREATE TABLE transaction_products (
373     transaction_id VARCHAR(50) NOT NULL,
374     product_id VARCHAR(20) NOT NULL,
375     PRIMARY KEY (transaction_id, product_id)
376 );
377
378 • ALTER TABLE transaction_products
379 ADD CONSTRAINT fk_tp_transactions
380 FOREIGN KEY (transaction_id) REFERENCES transactions(id);
381
382 • ALTER TABLE transaction_products
383 ADD CONSTRAINT fk_tp_products
384 FOREIGN KEY (product_id) REFERENCES products(id);
385
386 • INSERT INTO transaction_products (transaction_id, product_id)
387 SELECT
388     t.id,
389     jt.product_id
390 FROM transactions t
391 JOIN JSON_TABLE(
392     CONCAT('["', REPLACE(t.product_ids, ',', '","'), "']'),
393     '$[*]' COLUMNS (product_id VARCHAR(20) PATH '$')
394 ) jt
395 JOIN products p
396 ON p.id = jt.product_id
397 WHERE t.product_ids IS NOT NULL
398 AND t.product_ids <> '';
399

```

100% 27:398

Action Output

	Time	Action	Response
3	22:51:09	ALTER TABLE transaction_products ADD CONSTRAINT fk_tp_transactions FOREIGN KEY (transacio...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
4	22:51:13	ALTER TABLE transaction_products ADD CONSTRAINT fk_tp_products FOREIGN KEY (product_id)...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
5	22:51:25	INSERT INTO transaction_products (transaction_id, product_id) SELECT t.id, jt.product_id FRO...	100000 row(s) affected Records: 100000 Duplicates: 0 Warning

- Resolví la cantidad de productos vendidos utilizando la misma tabla “intermedia” que contiene el nro de producto y el de transaccional realizada.

399 -- -----| Respuesta de cant Vendidos

```

SELECT
  product_id AS producto,
  COUNT(*) AS veces_vendido
FROM transaction_products
    
```

100% 32:397

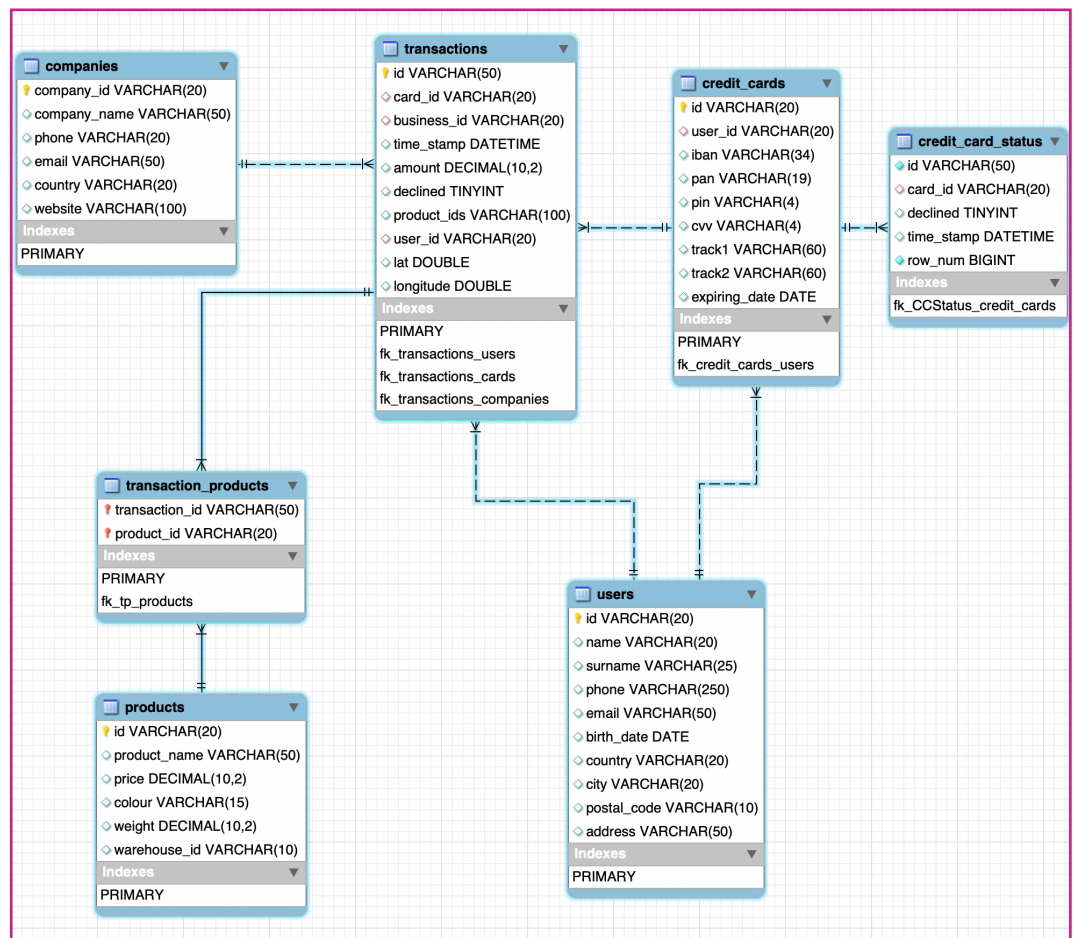
Result Grid Filter Rows: Search Export:

producto	veces_vendido
1	990
10	992
100	987
11	1048
12	994
13	1070
14	957
15	1026
16	1051
17	1014
18	979
19	971
2	1034

Result 54

Action Output

	Time	Action	Response
6	22:54:01	SELECT product_id AS producto, COUNT(*) AS veces_vendido FROM transaction_products GROUP...	100 row(s) returned



Modelo Relacional - esquema de Copo de Nieve

Está diseñado para mantener la integridad referencial, asegurando que no existan transacciones de usuarios o tarjetas que no existan en el sistema.

Relaciones y Tipos

El diagrama muestra relaciones de Entidad-Relación sólidas:

- Uno a Muchos (1:N): Un usuario puede tener muchas tarjetas; una empresa puede recibir muchas transacciones.
- Muchos a Muchos (M:N): Una transacción puede tener muchos productos y un producto estar en muchas transacciones. Has resuelto esto correctamente con la tabla intermedia transaction_products
- La Tabla de Hechos es transactions.
- ¿Por qué? Porque es la tabla que contiene los eventos cuantitativos (el "hecho" de la compra), las métricas (amount) y las claves foráneas que conectan con todo el contexto (quién compró, con qué tarjeta, en qué empresa y qué productos).

Las Tablas de Dimensiones

Las dimensiones son las que rodean a la tabla de hechos y le dan "contexto" a los números:

- users: Dimensión de clientes.
- companies: Dimensión de comercios.
- products: Dimensión de catálogo.
- credit_cards: Dimensión de métodos de pago.

P2P by: Tatiana Lukinax